

SafeSlice: Review 2 Code Walkthrough

Safe Dynamic Network Slicing for SLA-Preserving SDN: implementation so far (about 50 %)

Prithvi Raghu (24BCE2624) · Sumanta Kumar (24BCT0302) · Guide: Sasikala R · SCOPE, VIT

This guide explains, in easy words, the part of the code we are presenting in Review 2. It covers the full control loop: traffic in, network simulated, decision made, safety checked, decision applied. It runs end to end for a single run. Large-scale experiments, comparisons and results come in the next phase.

Contents

1. What we are showing in Review 2
 2. The big picture: how the pieces connect
 3. Module 1: Configuration
 4. Module 2: Traffic generator
 5. Module 3: Network interface
 6. Module 4: Network simulator
 7. Module 5: Context builder
 8. Module 6: Reward function
 9. Module 7: Policies and baselines
 10. Module 8: LinUCB, the learning agent
 11. Module 9: Safety guardrail
 12. Module 10: The control loop
 13. Testing
 14. Live demo script
 15. Next phase (Review 3)
 16. Questions faculty might ask about the code
 17. One-minute summary to speak
-

1. What we are showing in Review 2

Status	Part	Files
✓ Shown	Configuration and traffic scenarios	<code>config/default.yaml</code> , <code>config/scenarios/*.yaml</code> , <code>config_loader.py</code>
✓ Shown	Traffic generator	<code>traffic/traces.py</code>
✓ Shown	Network interface and simulator	<code>net/backend.py</code> , <code>net/sim_backend.py</code>
✓ Shown	Context (the 9 measurements)	<code>agent/context.py</code>
✓ Shown	Reward (scoring)	<code>agent/reward.py</code>
✓ Shown	Policy interface and simple baselines	<code>agent/policies/base.py</code> , <code>static.py</code> , <code>threshold.py</code>
✓ Shown	LinUCB learning agent	<code>agent/bandit.py</code> (the <code>LinUCB</code> class), <code>agent/policies/bandit_policy.py</code> (<code>LinUCBPolicy</code>)
✓ Shown	Safety guardrail	<code>agent/guardrail.py</code>
✓ Shown	Single-run control loop	<code>experiments/run_experiment.py</code>
✓ Shown	Unit tests for the above	<code>tests/test_sim_backend.py</code> , <code>test_guardrail.py</code> , <code>test_reward.py</code> , <code>test_bandit.py</code> , <code>test_threshold.py</code>
🕒 Next phase	Context-free bandit (ablation), pre-training, oracle	<code>EpsilonGreedy</code> in <code>agent/bandit.py</code> , <code>agent/policies/oracle.py</code>
🕒 Next phase	Full experiment suite, tuning, sensitivity study	<code>experiments/run_suite.py</code> , <code>tune.py</code> , <code>sensitivity.py</code>
🕒 Next phase	Analysis, tables, graphs, results	<code>analysis/*</code> , <code>results/</code>

Size: the shown part is about **2,100 of the roughly 4,000 lines of project code** (tests not counted), which is about half.

One-line status for the review: *"The complete control pipeline is implemented and tested: a traffic generator, a queue simulator of a 10 Mbps shared link, a 9-feature context, a reward function, a LinUCB contextual bandit, and a deterministic safety guardrail, all connected in a per-second control loop that runs end to end."*

2. The big picture: how the pieces connect



Three slices (queues), used everywhere in the code:

Queue index	Slice	Meaning
0	URLLC	Urgent, delay-sensitive traffic. We protect this.
1	eMBB	Heavy, high-bandwidth traffic. This is the slice whose limit we control.
2	Best Effort	Normal traffic. Gets a fixed limit.

3. Module 1: Configuration

Files: config/default.yaml , config/scenarios/*.yaml , config_loader.py

What it does: every number in the project lives in one YAML file, not scattered through the code. A scenario file (for example burst.yaml) is layered on top of the default settings.

Most important settings in default.yaml :

Setting	Value	Meaning
<code>link.capacity_bps</code>	10,000,000	The shared link is 10 Mbps
<code>run.duration_s / warmup_s</code>	300 / 30	Each run is 300 s; the first 30 s are ignored
<code>run.control_interval_s</code>	1.0	One decision per second
<code>action.embb_levels</code>	[0.20, 0.35, 0.50, 0.65, 0.80]	The 5 choices: eMBB may use 20 %-80 % of the link
<code>sla.target_ms / hard_ms</code>	5.0 / 7.0	Target and hard limit for URLLC delay
<code>guardrail.warn_ms</code>	4.0	Delay at which the guardrail starts restricting choices
<code>guardrail.holddown_steps</code>	3	Cool-down length after an emergency
<code>policy.linucb.alpha</code>	0.5	How much LinUCB explores
<code>reward.w_*</code>	1.0, 0.30, 1.0, 2.0, 0.50	Weights inside the score

`config_loader.py` in simple words:

- `load_config(scenario="burst")` reads `default.yaml`, merges `burst.yaml` on top (`_deep_merge`), and returns a `Cfg` object.
- `Cfg` lets the code write `cfg.link.capacity_bps` instead of `cfg["link"]["capacity_bps"]`.
- It is **read-only**, and a **typo raises an error** that shows the full key path. Settings can't be changed by accident, and a spelling mistake is never silently read as "empty".
- `overrides` let you change a setting from the command line, for example `--set run.duration_s=60`.
- `config_hash()` makes a short fingerprint of the settings, so we can prove which settings a run used.

Point to make: *"No magic numbers in the code: all parameters are in one config file, so experiments are easy to change and reproduce."*

4. Module 2: Traffic generator

File: `traffic/traces.py`

What it does: creates the **offered load**, meaning how much traffic each slice *wants* to send, second by second.

Scenario files describe phases. For example, `burst.yaml` says: 40 s of quiet eMBB (2 Mbps), then 35 s of heavy eMBB (10 Mbps), repeated. URLLC stays at 3 Mbps. A phase can also **ramp** smoothly from one value to another (`embb_mbps_end`).

Key functions:

Function	What it does in simple words
<code>_expand_phases()</code>	Turns the list of phases into a second-by-second schedule, repeating it until the run is 300 s long
<code>_lognormal_mean_preserving()</code>	Adds random wobble (jitter) to the traffic, so it looks realistic, without changing the average
<code>trace_seed()</code>	Turns (scenario name, seed number) into a random-number seed using blake2b hashing
<code>build_trace()</code>	Puts it all together and returns a <code>Trace</code> object with the traffic for every 10 ms time slice

Why this design matters:

- **Same seed means exactly the same traffic.** This is essential for fairness: every policy can be tested on identical traffic.
- We use **blake2b**, not Python's built-in `hash()`, because `hash()` gives different answers every time Python restarts. That would quietly break reproducibility.
- The jitter keeps the **average exactly equal** to the value written in the YAML file.

5. Module 3: Network interface

File: `net/backend.py`

What it does: defines the **rules any network must follow** to be controlled by our agent. It is an abstract class (`NetworkBackend`) with three methods:

```
class NetworkBackend(ABC):
    def read_telemetry(self) -> TelemetrySample: ... # advance 1 s, return measurements
    def apply_allocation(self, alloc: Allocation): ... # set the new rate limits
    def reset(self): ... # start a fresh run
```

Two data containers:

- `TelemetrySample` : what the network reports every second: URLLC delay (p50 and p95), throughput of each slice, drops per queue, link utilisation, and backlog per queue. It is **frozen** (cannot be edited after creation), so measurements can't be changed by mistake.
- `Allocation` : the rate limits to program into the network.

`allocation_from_level(cfg, level_index)` turns a choice (0–4) into actual limits: - URLLC is **never capped**. Capping the slice we protect would make no sense. - eMBB cap = chosen level × 10 Mbps (for example, level 2 gives $0.50 \times 10 = 5$ Mbps). - Best Effort cap is **fixed** at 45 % of the link. It is deliberately independent of the choice, so that raising eMBB genuinely takes capacity away from URLLC.

Point to make: *"The agent, guardrail and loop are written against this interface, not against the simulator. A real switch (Open vSwitch) can be plugged in later by writing one new class, without touching the agent."*

6. Module 4: Network simulator

File: `net/sim_backend.py` (the largest module in this review)

What it does: pretends to be the network. It simulates **three queues** sharing **one 10 Mbps link**, in small **10 ms time slices** (100 slices per 1-second decision).

The main idea: token buckets. Each queue has a "bucket" that fills with tokens at the queue's allowed rate. A queue can only send as many bytes as it has tokens. This is exactly how real switches (Open vSwitch HTB queues) enforce rate limits, so "cap eMBB at 50 %" means the same thing in our simulator as on a real switch.

What happens in one 10 ms slice (`_serve_one_substep`):

1. **Arrivals join the queue.** If the queue is full (62,500 bytes, about 50 ms of link capacity), the extra packets are **dropped** (tail drop).
2. **Tokens refill**, up to the bucket size.
3. **Each queue asks to send** `min(what is waiting, tokens available)`.
4. **The link shares its capacity:** - First, every queue gets its **guaranteed minimum** (URLLC 10 %, eMBB 10 %, BE 5 %). - The **leftover** is split by `_share_excess()`. By default, busier queues get a bigger share (`demand_proportional`). A heavy eMBB flow can therefore crowd out URLLC, which is the problem our controller must manage.
5. **URLLC delay is calculated:** `delay = base 2 ms + (URLLC bytes waiting ÷ drain rate)`.

Every 1 second (`read_telemetry`) it runs 100 slices, then reports the average throughputs, the **p50 and p95 delay** across those slices, the drops, the link utilisation and the backlog.

Good engineering details worth mentioning:

- **Bucket size is set per queue** from that queue's own limit (`_set_caps`). An early version used one fixed size for all queues, which made levels 50 %, 65 % and 80 % behave identically. We found and fixed that bug.
- **No strict priority for URLLC.** With strict priority, URLLC would never wait, delay would never rise, and there would be nothing to control. We use fair sharing with limits, like a real HTB switch.
- **Three sharing modes** are supported (`demand_proportional` , `equal` , `min_rate_proportional`), so the key modelling assumption can be changed and tested.
- **Drops are counted precisely:** fractional bytes are carried forward rather than rounded every slice.
- **Delay is counted once, not twice:** only one direction of the link is shaped. An early plan used 2×; we corrected it.

7. Module 5: Context builder

File: `agent/context.py`

What it does: turns raw telemetry into the **9 numbers** that the learning agent looks at before every decision.

#	Feature	Meaning	Divided by
1	<code>bias</code>	Always 1 (lets the model learn a base value per choice)	-
2	<code>rtt_ewma_norm</code>	Smoothed URLLC delay (slow, stable)	hard limit 7 ms
3	<code>rtt_p95_norm</code>	Right-now p95 URLLC delay (fast)	hard limit 7 ms
4	<code>urllc_util</code>	How much URLLC traffic is flowing	link capacity
5	<code>embb_util</code>	How much eMBB traffic is flowing	link capacity
6	<code>be_util</code>	How much Best Effort traffic is flowing	link capacity
7	<code>link_util</code>	How full the whole link is	-
8	<code>q0_backlog_norm</code>	URLLC bytes waiting in the queue	queue limit
9	<code>drops_norm</code>	Smoothed packet drops	packets per second the link can carry

Key ideas:

- **Normalisation:** every feature is scaled to roughly 0-1 (and clipped at 3). Without this, a number in bits per second (millions) would dominate one in milliseconds, simply because of its units.
- **EWMA** (smoothed value) = $0.3 \times \text{new value} + 0.7 \times \text{old smoothed value}$.
- **Both slow and fast delay are included on purpose.** The fast signal lets a policy react before the smoothed signal catches up.
- **URLLC utilisation is included on purpose.** It tells the agent that URLLC traffic is rising *before* delay goes up.
- The `Context` object also carries the delay **in milliseconds** for the guardrail. The safety component reads real units, not scaled numbers.

8. Module 6: Reward function

File: `agent/reward.py`

What it does: gives one **score** for each second, which tells the agent how good its last decision was.

```

score = 1.0 × (eMBB throughput / capacity)
      + 0.3 × (Best Effort throughput / capacity)
      - 1.0 × max(0, delay - 5 ms) / 5 ms
      - 2.0 × [1 if delay > 7 ms]
      - 0.5 × (drops / reference)

```

reward for heavy traffic served
 smaller reward for normal traffic
 penalty grows the more we miss the target
 big fixed penalty for breaking the hard limit
 penalty for dropped packets

Design points to explain:

- **p95 delay is used, not the average.** Averages hide the slow packets.
- **Throughput is divided by link capacity**, not by demand, because capacity is the resource we are sharing.
- **Best Effort has a real weight (0.3).** Otherwise the best answer would always be "give everything to eMBB".
- **The penalty has two parts: a sliding one and a jump.** Missing by 1 ms costs less than missing by 5 ms, and crossing the hard limit is always expensive.
- **The reward only sees the network's measurements.** It never sees which policy acted or what the guardrail did, so no policy can "game" the score.
- `reward_breakdown()` returns every term separately, so we can see which part drove a score.

9. Module 7: Policies and baselines

Files: `agent/policies/base.py`, `static.py`, `threshold.py`

The common interface (`base.py`). Every decision-maker is a `Policy` with:

```
select(ctx, allowed) -> int      # pick a choice 0..4, only from the allowed ones
update(ctx, action, reward)     # learn from the result (does nothing for fixed rules)
reset(seed)                     # start fresh for a new run
```

`allowed` is a list of True/False values, one per choice, supplied by the guardrail.

`nearest_allowed()` finds the closest allowed choice and, on a tie, picks the **safer (lower)** one.

The simple baselines (comparison points, built with the same interface):

Policy	Rule
<code>StaticSafe</code>	Always the lowest level (20 %). Very safe, wastes capacity.
<code>StaticEqual</code>	Always the level nearest a 1/3 equal split (35 %).
<code>Threshold</code>	If smoothed delay > 5 ms, step down one level; if < 3 ms, step up one level; otherwise stay.

A careful detail in `Threshold` : the current level is only changed in `update()`, using the level that was **actually applied**. `select()` can be called twice in one step (when the guardrail asks it to re-pick), so changing the level there would move it two steps by mistake.

Why `Threshold` only watches the smoothed delay: it represents a typical hand-written controller. Our learning agent's advantage is supposed to come from seeing more (the fast signal and URLLC load), so the comparison must keep that difference.

10. Module 8: LinUCB, the learning agent

Files: `agent/bandit.py` (class `LinUCB`), `agent/policies/bandit_policy.py` (class `LinUCBPolicy`)

The idea in simple words

Imagine five slot machines (our five eMBB levels). How much each machine pays depends on the situation (the 9-number context). For **each machine**, LinUCB keeps a small **linear model** that predicts the score from the 9 numbers. Before each decision it computes, for every machine:

$$\text{UCB score} = \text{predicted score} + \alpha \times \text{uncertainty}$$

and picks the highest. The **uncertainty bonus** is large for choices it hasn't tried much in situations like this one. That makes it **explore** them; as it gathers data, the bonus shrinks and it **exploits** what it knows.

The maths (short version)

For each choice a , it stores a 9×9 matrix $\mathbf{A}$ and a 9-number vector $\mathbf{b}$:

- $\mathbf{A} = \mathbf{I} + \sum \mathbf{x} \mathbf{x}^T$ (built up from every context $\mathbf{x}$ in which a was played)
- $\mathbf{b} = \sum \text{reward} \times \mathbf{x}$
- weights $\boldsymbol{\theta} = \mathbf{A}^{-1} \mathbf{b}$, prediction $\boldsymbol{\theta} \cdot \mathbf{x}$, uncertainty $\sqrt{(\mathbf{x}^T \mathbf{A}^{-1} \mathbf{x})}$

How the code does it

Method	What it does
<code>reset()</code>	Start with $\mathbf{A}^{-1} = \text{identity}$ and $\mathbf{b} = \mathbf{0}$ for all 5 choices
<code>scores(x, allowed)</code>	Computes the UCB score of all 5 choices at once (<code>np.einsum</code>); disallowed choices get $-\infty$
<code>select(x, allowed)</code>	Picks the highest score; ties go to the lower, safer level
<code>update(x, action, reward)</code>	Updates $\mathbf{A}^{-1}$ with the Sherman-Morrison formula and adds $\text{reward} \times \mathbf{x}$ to $\mathbf{b}$

Sherman-Morrison updates the inverse matrix directly with a cheap formula, instead of re-inverting the matrix every second. It is fast and is what a real deployment would use. A test checks it against an explicit matrix inverse after many updates, so rounding errors cannot build up unnoticed.

`LinUCBPolicy` is a thin adapter: it feeds `ctx.vector` (the 9 numbers) into `LinUCB`, and keeps the guardrail's allowed list respected. `alpha = 0.5` comes from the config file.

Why it's lightweight: each decision is a few small 9×9 matrix operations, which takes a fraction of a millisecond. Deep reinforcement learning, by contrast, needs a neural network and long training.

11. Module 9: Safety guardrail

File: `agent/guardrail.py`

What it does: stands **between** the policy and the network. Every decision passes through `Guardrail.apply(ctx, proposed_action, policy)`.

Checks, in priority order:

Order	State	Condition	Action
1	<code>holddown</code>	A cool-down is still running	Force the safest level (20 %)
2	<code>hard_override</code>	Smoothed or right-now delay > 7 ms	Force the safest level and start a 3-step cool-down
3	<code>warn_mask_pass</code> / <code>warn_mask_reselect</code>	Smoothed or right-now delay > 4 ms	Allow only levels 0-2 (20-50 %). If the policy's pick is not allowed, ask the policy to re-pick from the allowed ones.
4	<code>ok</code>	Everything is normal	Allow the policy's choice

Important design points:

- **It asks rather than overrules.** In the warning zone the policy chooses again among the safe options, so the policy's knowledge about which safe option is best is still used.
- **A final clamp (`_finish`)** guarantees the applied choice is always inside the allowed set, even if a policy has a bug. That guarantee lives in this one file.
- **It checks both the slow and the fast delay.** The smoothed value alone reacts too late to sudden spikes. `escalation_source` records which signal triggered it (`ewma` , `p95` or `both`).
- **Intervention is counted honestly:** only when the applied choice differs from the proposed one.
- **Settings are checked at start-up.** For example, the warning level must be below the hard limit, or the program refuses to start.

12. Module 10: The control loop

File: `experiments/run_experiment.py` (function `run_once`)

What it does: connects everything and runs one experiment for a given (policy, scenario, seed). The heart of it:

```

ctx = ctx_builder.update(backend.idle_sample())           # starting context

for step in range(n_steps):                             # one loop = one second
    proposed = policy.select(ctx, unrestricted)          # 1. policy proposes
    decision = guardrail.apply(ctx, proposed, policy)    # 2. guardrail checks
    alloc = allocation_from_level(cfg, decision.applied_action)
    backend.apply_allocation(alloc)                     # 3. apply the new eMBB cap
    tel = backend.read_telemetry()                      # 4. 1 second passes, measure
    rb = reward_breakdown(tel, cfg)                    # 5. score it
    policy.update(ctx, decision.applied_action, rb.total) # 6. learn
    rows.append({...})                                  # 7. log everything
    ctx = ctx_builder.update(tel)                       # 8. context for next second

```

The order matters (point this out):

- The decision uses measurements from the **previous** second only, just like a real controller. Using the current second would be "seeing the future".
- The score is credited to the choice that was **actually applied** (after the guardrail), so the agent learns about the right choice.

Outputs of one run:

- A **CSV log with one row per second**: traffic offered, delay, throughput, drops, all 9 context values, the proposed choice, the applied choice, the guardrail's reason, decision time and every reward term.
- An `env.json` file recording the Python version, computer, git commit and settings hash, so the run can be reproduced.
- The CSV is **checked against the expected column layout** after writing.

It also measures **decision time** for every step with `time.perf_counter()`.

13. Testing

We wrote automated tests with **pytest** for every module in this review:

Test file	What it checks (examples)
<code>tests/test_sim_backend.py</code>	Rate caps really limit traffic; higher eMBB levels raise URLLC delay; the sharing mode changes the outcome; same seed gives the same result
<code>tests/test_guardrail.py</code>	Hard override and cool-down work; re-pick works; the applied choice is always inside the allowed set , checked over 5,000 random cases with a "bad" policy that always asks for the riskiest level
<code>tests/test_reward.py</code>	Each reward term behaves as intended (penalties, weights)
<code>tests/test_bandit.py</code>	LinUCB learns the right answer on a textbook problem; Sherman-Morrison matches a real matrix inverse
<code>tests/test_threshold.py</code>	Threshold steps up and down correctly and does not double-step

Command: `pytest -q tests/test_sim_backend.py tests/test_guardrail.py tests/test_reward.py tests/test_bandit.py tests/test_threshold.py` gives **68 passed** (in about 5 seconds).

14. Live demo script

A short run, about 5 seconds of computer time, to show the loop working:

```
python experiments/run_experiment.py --policy linucb --scenario sawtooth --seed 0 \
  --set run.duration_s=60 --set run.warmup_s=0
```

Then open the CSV it writes (`results/raw/linucb_sawtooth_0.csv`) and show these columns. This is a real excerpt from that command (URLLC load in Mbps, delay in ms):

step	URLLC load	URLLC p95 delay	proposed	applied	guardrail reason
0	1.46	2.00	0	0	ok
2	1.60	2.00	1	1	ok
3	1.50	2.00	2	2	ok
7	1.46	2.00	4	4	ok
30	3.01	2.00	4	4	ok
34	2.98	4.14	4	4	ok
35	2.76	3.70	4	2	warn_mask_reselect
36	2.86	8.19	4	4	ok
37	3.12	2.00	3	0	hard_override
38	2.92	2.00	3	0	holddown
39	2.92	2.00	3	0	holddown

What to say while showing it:

1. **Steps 0-7:** LinUCB starts at the safest level and **explores upward** ($0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 4$) while the delay is fine. This is the uncertainty bonus at work.
2. **Step 30:** the URLLC load jumps from about 1.5 to about 3 Mbps (a new phase of the scenario).
3. **Step 35:** delay in the previous second was above 4 ms, so the guardrail **allowed only levels 0-2**. LinUCB wanted 4, so it was **asked to re-pick** and chose 2.
4. **Step 36:** the previous second's signals had dropped back below 4 ms, so there were no restrictions. LinUCB picked 4 and delay jumped to 8.19 ms, crossing the 7 ms limit.
5. **Step 37:** the guardrail saw that breach and applied the **emergency brake** (level 0).
Steps 38-39: the **cool-down** holds it there, even though LinUCB still wants level 3. Delay returns to 2 ms.

This also shows honestly what the guardrail can and can't do. It reacts to the previous second, so it cannot stop the first second of a sudden jump (step 36). It then stops the problem continuing. (By default the run writes into `results/raw/`. Add `--out-dir some_folder` to put it elsewhere.)

The run also prints a short summary at the end. For Review 2 you can simply say *"the run completes and the log is validated"* and focus on the table above.

15. Next phase (Review 3)

Work item	Purpose
Context-free bandit (ϵ -greedy)	Check whether the 9-feature context actually helps
Pre-training on separate seeds	Measure LinUCB after it has learned, not just while learning
Oracle reference	A "best possible fixed plan" line to compare against
Full experiment suite (8 policies $\times$ 4 scenarios $\times$ 10 seeds)	Fair, large-scale comparison
Hyperparameter tuning and reward-weight sensitivity	Make sure conclusions don't depend on one setting
Analysis, tables and graphs	Present the results

16. Questions faculty might ask about the code

Q: Why did you write your own simulator instead of using Mininet? A: Our development machine had no Mininet, Open vSwitch or root access. The simulator uses the same token-bucket mechanism as Open vSwitch HTB queues, so the actions mean the same thing. The code talks to a `NetworkBackend` interface, so a real switch backend can be added later without changing the agent.

Q: Why 5 fixed levels and not "increase / decrease" actions? A: With increase/decrease, the effect of an action depends on where you started, so the problem stops being a bandit problem. A fixed level means the same thing every time.

Q: How does LinUCB decide? A: For each of the 5 levels it keeps a linear model that predicts the score from the 9 context numbers, adds an uncertainty bonus ($\alpha \times \sqrt{(x^T A^{-1} x)}$), and picks the highest.

Q: What is Sherman-Morrison and why use it? A: It is a formula for updating a matrix inverse after adding one data point, without inverting from scratch. It keeps each decision very cheap.

Q: What if LinUCB picks something dangerous? A: The guardrail sits in between. Above 4 ms it only allows levels up to 50 %, and above 7 ms it forces the safest level with a 3-second cool-down. A final clamp guarantees the applied choice is always allowed, and a test checks this over 5,000 random cases.

Q: Can the guardrail prevent every violation? A: No. It acts on the previous second's measurements, so the very first second of a sudden spike can still go over. The demo shows exactly this (step 36), and then the guardrail stopping it.

Q: How do you make runs reproducible? A: The traffic is generated from a seed using blake2b hashing, so the same seed gives exactly the same traffic. All settings are in one config file, and every run saves an `env.json` with the settings hash and git commit.

Q: Why are the delay limits 5 / 7 ms? A: We measured the delay at each level in the simulator (about 2 ms at 20 % up to about 9 ms at 80 %) and placed the limits between the middle levels, so the choice actually matters.

Q: How do you know the code is correct? A: 68 automated tests cover the simulator, guardrail, reward, LinUCB and threshold policy.

17. One-minute summary to speak

"Our project controls how a 10 Mbps link is shared between three 5G-style slices: urgent URLLC, heavy eMBB and Best Effort. So far we have implemented the complete control pipeline in Python. A traffic generator creates reproducible traffic scenarios. A token-bucket queue simulator, modelled on Open vSwitch queues, simulates the link in 10 ms steps. Every second, we turn the measurements into 9 normalised features, and a LinUCB contextual bandit picks one of five bandwidth limits for eMBB. Before that choice is applied, a deterministic safety guardrail checks it: it limits choices when delay passes 4 ms and forces the safest level when it passes 7 ms. A reward function scores each second, and LinUCB learns from it. The loop runs end to end and logs every decision, and 68 unit tests cover these modules. In the next phase we'll run the full experiment suite, compare against baselines, and present the results."